

INSTITUCIÓN UNIVERSITARIA

PASCUAL BRAVO®

INSTITUCIÓN UNIVERSITARIA
PASCUAL BRAVO®

Fecha: 25/10/2026

Estudiante: Daniel Steven Pineda Zafra

Docente: Oralia Cortes Grajales

Materia: Bases de Datos I

Tarea. Comparar la implementación de un diccionario de datos físico en diferentes SGBD.

Implementación del modelo físico

La Red Social Pascualina busca apoyar la comunicación y la interacción entre estudiantes. El diseño contempla perfiles, intereses, habilidades, deportes, seguimiento, mentorías, publicaciones, grupos y eventos. Esta etapa no crea una aplicación web: implementa la estructura que permitirá almacenar y relacionar esa información.

El objetivo fue transformar el modelo lógico en una base de datos funcional mediante DDL, definiendo tablas, tipos, claves, restricciones y acciones referenciales. La construcción partió de estudiante, interes, publicacion y estudiante_interes, desarrolladas en el foro, y se amplió sin sustituir el trabajo anterior.

Inventario

Componente	Tablas implementadas	Cantidad
Perfil y catálogos	estudiante, interes, habilidad, deporte	4
Contenido y actividades	publicacion, grupo, evento	3
Relaciones entre estudiantes	conexion, mentoria	2
Tablas asociativas	estudiante_interes, estudiante_habilidad, actividad_fisica, ingreso_grupo, asistencia	5

El modelo contiene 14 claves primarias: ocho simples y seis compuestas. Incluye 17 claves foráneas, una restricción UNIQUE sobre el correo institucional y cuatro restricciones CHECK explícitas. Los campos obligatorios se controlan con NOT NULL o por su pertenencia a la PK.

1. Elección del SGBD

Se seleccionó PostgreSQL 18. Su arquitectura cliente-servidor y sus mecanismos de integridad permiten centralizar los datos y expresar las relaciones del modelo. La elección también aprovecha el entorno ya utilizado en el foro, evitando rehacer una implementación que se estaba comprendiendo y validando

Comparación con dos alternativas

SGBD	Ventajas para este caso	Desventajas o ajustes necesarios
PostgreSQL 18	Permite definir PK, FK, UNIQUE y CHECK. Ofrece UUID nativo y columnas IDENTITY, utilizados en este proyecto. Su arquitectura admite conexiones de varios clientes.	Requiere instalar y administrar un servidor, sus conexiones y permisos. Para un ejercicio local muy pequeño supone más configuración que una base embebida.
MySQL 8.4 con InnoDB	Es una alternativa viable para conservar relaciones entre estudiantes, publicaciones y membresías. InnoDB admite claves foráneas y acciones referenciales; MySQL 8.4 también aplica CHECK.	El script no se trasladaría sin ajustes: habría que adaptar la generación de identificadores y comprobar las diferencias de sintaxis. Además, los nombres de FK deben ser únicos en la base, lo que afecta nombres reutilizados como fk1_estudiante.
SQLite	Es una opción sencilla para prototipos locales: funciona integrada en la aplicación y no necesita un proceso servidor separado. También dispone de soporte de FK.	Solo permite un escritor simultáneo por archivo de base de datos. Para una red social con muchas escrituras concurrentes conviene evaluar un servidor. Las FK deben habilitarse en cada conexión según su configuración.

PostgreSQL ofrece una buena correspondencia con las decisiones de este proyecto: claves compuestas, relaciones recursivas, fechas, restricciones y generación de identificadores. MySQL también podría resolver el caso; SQLite sería especialmente práctico para un prototipo local. La elección no se basa en afirmar que un motor sea siempre superior ni en una prueba de velocidad, sino en el ajuste al modelo y la continuidad del aprendizaje.

2. Del modelo lógico al físico

Las catorce tablas conservan las responsabilidades definidas en la etapa lógica. Durante la implementación se concretaron sus tipos de datos, mecanismos de generación y restricciones. Los siguientes cambios explican las diferencias entre el informe anterior y el script actual.

Elemento	Diseño previo	Implementación física y razón
interes, habilidad, deporte y mentoria	ID VARCHAR(12)	INTEGER GENERATED ALWAYS AS IDENTITY. El motor genera el identificador y evita asignarlo manualmente.
publicacion, grupo y evento	ID VARCHAR(12)	UUID DEFAULT gen_random_uuid(). Se adoptan identificadores automáticos de formato UUID y se mantiene la PK.
Columnas foráneas	Referencia a ID VARCHAR(12)	Se ajustan a INTEGER o UUID según el nuevo tipo de la tabla padre. Las FK no generan IDs nuevos.
evento.fecha_hora_evento	DATETIME	TIMESTAMP, tipo disponible en PostgreSQL para almacenar fecha y hora sin zona horaria.
asistencia.fecha_respuesta	DATE	TIMESTAMP para conservar también la hora. Sigue siendo obligatorio y no tiene DEFAULT en el script.
Integridad adicional	Claves y obligatoriedad	CHECK del semestre, prohibición de autoseguimiento, mentor distinto del aprendiz y orden de fechas de mentoría.
Datos opcionales	Opcionales en el modelo	Se conservan numero_contacto, descripciones de grupo/evento, frecuencia, nivel_habilidad y fecha_fin_mentoria.

Se mantienen nombres descriptivos, sin espacios ni tildes en los identificadores y con guión bajo para separar palabras. El script mezcla mayúsculas y minúsculas, pero no usa comillas dobles en los nombres; PostgreSQL los interpreta en minúsculas. Los nombres reales se documentan aquí de esa manera.

Los comentarios separan atributos y reglas. Las restricciones conservan los nombres que se construyeron durante el ejercicio. La PK y las FK se declaran al final de cada tabla para facilitar su lectura, no porque ese orden cambie su comportamiento.

3. Criterios de tipos e integridad

Identificadores

La cédula se conserva como VARCHAR(12): identifica al estudiante y no representa una cantidad. Los IDs de catálogos y mentorías usan INTEGER con IDENTITY.

En publicaciones, grupos y eventos se utiliza UUID nativo. gen_random_uuid() produce un UUID aleatorio; la PK es la restricción que impide duplicados. Las columnas FK reciben el ID existente y no llevan IDENTITY ni generación UUID.

Texto, fechas y valores lógicos

Se conserva VARCHAR(n) para imponer los límites de longitud del diseño. VARCHAR(20000) limita el contenido de una publicación a 20.000 caracteres.

DATE se utiliza cuando solo interesa el día; TIMESTAMP, cuando también interesa la hora. BOOLEAN representa privacidad y respuesta. La convención es TRUE = privado y FALSE = público; en asistencia, TRUE = confirma y FALSE = no confirma.

Restricción	Condición o efecto en el proyecto
uq_correo	No permite repetir correo_institucional.
ck_semestre	semestre BETWEEN 1 AND 20. Es una regla adoptada para el ejercicio, no un límite oficial de la institución.
ck_no_autoseguimiento	cedula_seguidor < > cedula_seguido.
ck_mentoria	cedula_mentor < > cedula_aprendiz.
ck_fechas_mentoria	La fecha de fin puede ser NULL; cuando existe, debe ser igual o posterior al inicio.

4. Descripción de las tablas

Tabla estudiante

Información personal y académica de cada estudiante.

Atributo	Tipo y tamaño	Reglas	Contenido
cedula	VARCHAR(12)	PK; Obligatorio	Identificación del estudiante.
nombre	VARCHAR(25)	Obligatorio	Nombre del estudiante.
apellidos	VARCHAR(25)	Obligatorio	Apellidos tratados como un dato textual.
correo_institucional	VARCHAR(50)	UNIQUE; Obligatorio	Correo institucional.
carrera	VARCHAR(50)	Obligatorio	Programa académico.
semestre	INTEGER	Obligatorio	Semestre actual.
numero_contacto	VARCHAR(15)	Opcional	Teléfono de contacto.

PK: cedula. Se eligió cedula como clave natural, manteniendo el modelo anterior. VARCHAR(12) permite conservar el identificador como texto. El correo es único; semestre es INTEGER con CHECK entre 1 y 20. numero_contacto permanece opcional.

interes

Catálogo de intereses que se pueden asociar a los estudiantes.

Atributo	Tipo y tamaño	Reglas	Contenido
id_interes	INTEGER	PK; Obligatorio	Identificador del interés. IDENTITY.
nombre_interes	VARCHAR(20)	Obligatorio	Nombre del interés.

PK: id_interes. id_interes identifica el registro independientemente de su nombre. Se cambió a INTEGER con IDENTITY para generar el ID automáticamente; nombre_interes conserva el límite de 20 caracteres.

publicación

Contenido publicado por un estudiante, con su autor y fecha.

Atributo	Tipo y tamaño	Reglas	Contenido
id_publicacion	UUID	PK; Obligatorio	Identificador de la publicación. DEFAULT gen_random_uuid().
contenido	VARCHAR(20000)	Obligatorio	Contenido textual.
cedula_autor	VARCHAR(12)	FK; Obligatorio	Estudiante que publica.
fecha_publicacion	DATE	Obligatorio	Día de la publicación. DEFAULT CURRENT_DATE.

PK: id_publicacion. id_publicacion usa UUID generado por defecto. La PK identifica cada publicación; cedula_autor es VARCHAR(12), como estudiante.cedula. Se conserva VARCHAR(20000) para limitar el contenido y DATE porque aquí solo se registró el día.

estudiante_interes

Asociaciones entre estudiantes y los intereses seleccionados.

Atributo	Tipo y tamaño	Reglas	Contenido
cedula	VARCHAR(12)	PK comp.; FK; Obligatorio	Identificación del estudiante.
id_interes	INTEGER	PK comp.; FK; Obligatorio	Identificador del interés.

PK: cedula, id_interes. La PK (cedula, id_interes) evita repetir una asociación. Cada columna puede repetirse por separado. El ID del interés es INTEGER y no autoincremental en esta tabla: referencia un interés existente.

habilidad

Catálogo de habilidades que pueden registrar los estudiantes.

Atributo	Tipo y tamaño	Reglas	Contenido
id_habilidad	INTEGER	PK; Obligatorio	Identificador de la habilidad. IDENTITY.
nombre_habilidad	VARCHAR(25)	Obligatorio	Nombre de la habilidad.

PK: id_habilidad. La PK es id_habilidad, generada con INTEGER IDENTITY. Se separa del nombre para que la referencia a la habilidad no dependa de su texto descriptivo. nombre_habilidad es obligatorio y admite hasta 25 caracteres

deporte

Catálogo de deportes y su clasificación general.

Atributo	Tipo y tamaño	Reglas	Contenido
id_deporte	INTEGER	PK; Obligatorio	Identificador del deporte. IDENTITY.
nombre_deporte	VARCHAR(25)	Obligatorio	Nombre del deporte.
tipo_deporte	VARCHAR(25)	Obligatorio	Clasificación del deporte.

PK: id_deporte. id_deporte es INTEGER IDENTITY. nombre_deporte y tipo_deporte conservan VARCHAR(25). tipo_deporte se registra como una clasificación textual; el script no impone un catálogo cerrado de valores.

grupo

Grupos académicos o sociales creados por estudiantes.

Atributo	Tipo y tamaño	Reglas	Contenido
id_grupo	UUID	PK; Obligatorio	Identificador del grupo. DEFAULT gen_random_uuid().
cedula_creador	VARCHAR(12)	FK; Obligatorio	Estudiante creador.
nombre_grupo	VARCHAR(50)	Obligatorio	Nombre del grupo.
descripcion	VARCHAR(200)	Opcional	Descripción del propósito.

fecha_creacion	DATE	Obligatorio	Día de creación. DEFAULT CURRENT_DATE.
privacidad	BOOLEAN	Obligatorio	TRUE privado; FALSE público.

PK: id_grupo. id_grupo utiliza UUID. cedula_creador identifica al creador, no a todos los integrantes. descripcion es opcional. La privacidad debe proporcionarse: el script no define un valor por defecto.

ingreso_grupo

Membresías de estudiantes en grupos, con fecha de ingreso y rol.

Atributo	Tipo y tamaño	Reglas	Contenido
cedula	VARCHAR(12)	PK comp.; FK; Obligatorio	Identificación del estudiante.
id_grupo	UUID	PK comp.; FK; Obligatorio	Identificador del grupo.
fecha_ingreso	DATE	Obligatorio	Día de ingreso al grupo. DEFAULT CURRENT_DATE.
rol	VARCHAR(25)	Obligatorio	Función del integrante.

PK: cedula, id_grupo. La PK (cedula, id_grupo) identifica una membresía y evita duplicarla. id_grupo es UUID como en grupo. rol y fecha_ingreso pertenecen a la relación; el rol es obligatorio, pero no tiene una lista de valores restringida por CHECK.

evento

Actividades organizadas por estudiantes con lugar y fecha programada.

Atributo	Tipo y tamaño	Reglas	Contenido
id_evento	UUID	PK; Obligatorio	Identificador del evento. DEFAULT gen_random_uuid().
cedula_creador	VARCHAR(12)	FK; Obligatorio	Estudiante creador.
nombre_evento	VARCHAR(50)	Obligatorio	Nombre del evento.
descripcion	VARCHAR(200)	Opcional	Descripción del propósito.
fecha_hora_evento	TIMESTAMP	Obligatorio	Día y hora programados.
lugar_evento	VARCHAR(100)	Obligatorio	Ubicación del evento.

PK: id_evento. La PK id_evento es UUID. El creador se referencia por su cédula. fecha_hora_evento usa TIMESTAMP para registrar día y hora; debe suministrarse y no recibe automáticamente la fecha actual.

conexion

Seguimiento dirigido entre dos estudiantes.

Atributo	Tipo y tamaño	Reglas	Contenido
cedula_seguidor	VARCHAR(12)	PK comp.; FK; Obligatorio	Estudiante que sigue.
cedula_seguido	VARCHAR(12)	PK comp.; FK; Obligatorio	Estudiante seguido.

fecha_conexion	DATE	Obligatorio	Día del seguimiento. DEFAULT CURRENT_DATE.

PK: cedula_seguidor, cedula_seguido. La PK (cedula_seguidor, cedula_seguido) evita repetir el mismo seguimiento. Las dos columnas son FK a estudiante, con roles distintos. CHECK impide el autoseguimiento; seguir a otra persona no crea automáticamente la conexión inversa.

mentoria

Orientación académica entre un mentor y un aprendiz.

Atributo	Tipo y tamaño	Reglas	Contenido
id_mentoria	INTEGER	PK; Obligatorio	Identificador de la mentoría. IDENTITY.
cedula_mentor	VARCHAR(12)	FK; Obligatorio	Estudiante mentor.
cedula_aprendiz	VARCHAR(12)	FK; Obligatorio	Estudiante aprendiz.
fecha_inicio_mentoria	DATE	Obligatorio	Día de inicio. DEFAULT CURRENT_DATE.
fecha_fin_mentoria	DATE	Opcional	Día de finalización.
descripcion	VARCHAR(200)	Obligatorio	Descripción del propósito.

PK: id_mentoria. id_mentoria es INTEGER IDENTITY y permite registrar varias mentorías entre la misma pareja. Las dos cédulas son FK obligatorias. fecha_fin_mentoria queda opcional; los CHECK impiden auto-mentoría y fechas finales anteriores al inicio.

asistencia

Respuesta de un estudiante a un evento: confirma o no confirma asistencia.

Atributo	Tipo y tamaño	Reglas	Contenido

cedula	VARCHAR(12)	PK comp.; FK; Obligatorio	Identificación del estudiante.
id_evento	UUID	PK comp.; FK; Obligatorio	Identificador del evento.
respuesta	BOOLEAN	Obligatorio	TRUE confirma; FALSE no confirma.
fecha_respuesta	TIMESTAMP	Obligatorio	Día y hora de la respuesta.

PK: cedula, id_evento. La PK (cedula, id_evento) conserva una respuesta por estudiante y evento. id_evento es UUID. fecha_respuesta se definió como TIMESTAMP obligatorio, sin DEFAULT, por lo que debe enviarse al registrar la respuesta.

actividad_fisica

Deportes asociados a cada estudiante y su frecuencia de práctica.

Atributo	Tipo y tamaño	Reglas	Contenido
cedula	VARCHAR(12)	PK comp.; FK; Obligatorio	Identificación del estudiante.
id_deporte	INTEGER	PK comp.; FK; Obligatorio	Identificador del deporte.
frecuencia	VARCHAR(25)	Opcional	Frecuencia de práctica.

PK: cedula, id_deporte. La PK (cedula, id_deporte) impide repetir el mismo deporte para un estudiante. id_deporte es INTEGER como en el catálogo. frecuencia se conserva opcional y textual, con un máximo de 25 caracteres.

estudiante_habilidad

Habilidades de cada estudiante y, opcionalmente, su nivel de dominio.

Atributo	Tipo y tamaño	Reglas	Contenido
cedula	VARCHAR(12)	PK comp.; FK; Obligatorio	Identificación del estudiante.
id_habilidad	INTEGER	PK comp.; FK; Obligatorio	Identificador de la habilidad.
nivel_habilidad	VARCHAR(20)	Opcional	Nivel de dominio.

PK: cedula, id_habilidad. La PK (cedula, id_habilidad) identifica cada asociación. id_habilidad referencia el catálogo INTEGER; nivel_habilidad describe esa habilidad para ese estudiante y no es obligatorio.

5. Relaciones y acciones referenciales

Cada fila documenta una FK real. La tabla padre contiene la clave referenciada; la tabla hija almacena la FK. Todas estas relaciones son 1:N desde el padre: un registro padre puede no tener hijos o tener varios, y cada hijo referencia exactamente un padre. Las FK son obligatorias en este modelo.

Las 17 FK usan ON UPDATE CASCADE para propagar una corrección de la clave referenciada, no cualquier cambio de la tabla padre. No se plantea cambiar IDs habitualmente. ON DELETE se decide por relación: 15 CASCADE y 2 RESTRICT.

FK en tabla hija	PK en tabla padre	ON DELETE	Justificación
publicacion. cedula_autor	estudiante. cedula	CASCADE	Las publicaciones dependen del autor y se retiran cuando se elimina su cuenta.
grupo. cedula_creador	estudiante. cedula	**RESTRICT**	Se protege el grupo y a sus demás integrantes; el creador no puede borrarse mientras exista el grupo a su cargo.
evento. cedula_creador	estudiante. cedula	**RESTRICT**	Se conserva el evento y las respuestas de otros estudiantes mientras tenga ese creador.
conexion. cedula_seguidor	estudiante. cedula	CASCADE	Se retiran seguimientos que ya no tienen seguidor.
conexion. cedula_seguido	estudiante. cedula	CASCADE	Se retiran seguimientos hacia una cuenta eliminada.
mentoria. cedula_mentor	estudiante. cedula	CASCADE	Se elimina la mentoría si desaparece el mentor. No se conserva historial de ese registro.
mentoria. cedula_aprendiz	estudiante. cedula	CASCADE	Se elimina la mentoría si desaparece el aprendiz. No se conserva historial de ese registro.
estudiante_interes. cedula	estudiante. cedula	CASCADE	Se retiran asociaciones de la cuenta; no se elimina el catálogo de intereses.
estudiante_interes. id_interes	interes. id_interes	CASCADE	Se retiran asociaciones del interés eliminado; no se borran estudiantes.
estudiante_habilidad. cedula	estudiante. cedula	CASCADE	Se retiran habilidades asociadas a la cuenta; no se borra el catálogo.

estudiante_habilidad. id_habilidad	habilidad. id_habilidad	CASCADE	Se retiran asociaciones de la habilidad eliminada; no se borran estudiantes.

FK en tabla hija	PK en tabla padre	ON DELETE	Justificación
actividad_fisica. cedula	estudiante. cedula	CASCADE	Se retiran asociaciones deportivas de la cuenta; no se borran deportes.
actividad_fisica. id_deporte	deporte. id_deporte	CASCADE	Se retiran asociaciones del deporte eliminado; no se borran estudiantes.
ingreso_grupo. cedula	estudiante. cedula	CASCADE	Se retiran membresías del estudiante, no los grupos de otras personas.
ingreso_grupo. id_grupo	grupo. id_grupo	CASCADE	Al desaparecer el grupo dejan de existir sus membresías; los estudiantes permanecen.
asistencia. cedula	estudiante. cedula	CASCADE	Se retiran respuestas del estudiante; no se elimina el evento.
asistencia. id_evento	evento. id_evento	CASCADE	Se retiran respuestas de un evento eliminado; los estudiantes permanecen.

Relaciones N:M y claves compuestas

Las asociaciones estudiante-interes, estudiante-habilidad, estudiante-deporte, estudiante-grupo y estudiante-evento se implementan mediante tablas puente. Cada una contiene dos FK y una PK compuesta: la pareja completa no puede repetirse, aunque cada valor individual sí. conexion aplica la misma idea a dos estudiantes, diferenciados por su rol.

Crear un grupo no es pertenecer a él

grupo.cedula_creador identifica a un creador. ingreso_grupo relaciona cada grupo con sus miembros. Por tanto, grupo tiene una relación 1:N con ingreso_grupo; la relación N:M ocurre entre estudiante y grupo a través de esa tabla. El script no agrega al creador automáticamente como miembro ni le asigna un rol. Esa operación debe registrarse por separado.

Efecto de RESTRICT

Si un estudiante conserva grupos o eventos creados, su eliminación será rechazada. Las cascadas de otras tablas no anulan esa protección. Primero se debe resolver la responsabilidad del grupo o evento conforme a la regla del sistema.

6 Estructura y ejecución del script DDL

El archivo red_social_pascualina.sql es la referencia ejecutable del informe. Conserva el orden y los comentarios construidos durante el proyecto, incluido el encabezado del foro como antecedente. Contiene 14 sentencias CREATE TABLE; no incluye instrucciones DROP, datos de prueba ni los ALTER TABLE que se utilizaron durante los ajustes.

Construcción desde cero

1. Abrir Query Tool conectado a la base red_social_pascualina vacía y comprobar la conexión con SELECT current_database();.

2. Abrir red_social_pascualina.sql y ejecutar el script completo. No repetirlo sobre tablas ya creadas: este archivo es de construcción inicial, no de migración.

3. Actualizar Schemas → public → Tables y verificar las catorce tablas. Revisar los mensajes y las restricciones antes de generar el ERD.

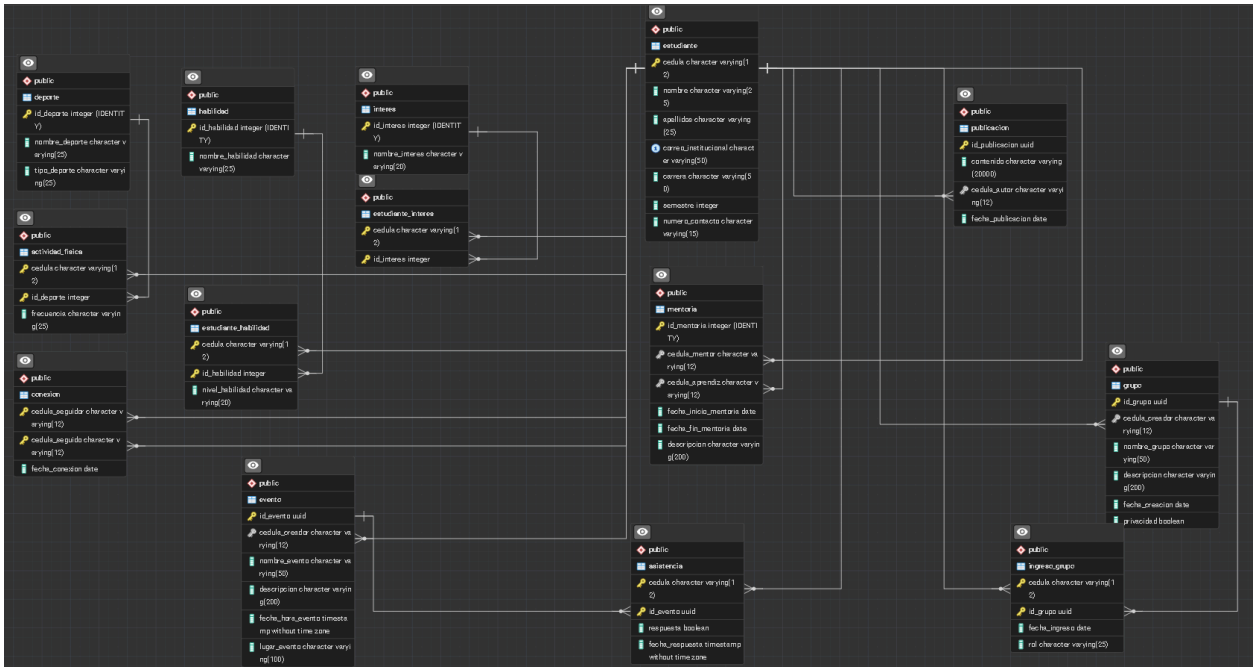
4. Para cambios posteriores, preparar ALTER TABLE específicos y conservar también una versión coherente del script de creación. Guardar texto SQL no cambia por sí solo la base de datos.

Consulta de comprobación utilizada

```
SELECT COUNT(*) AS total_tablas
FROM information_schema.tables
WHERE table_schema = 'public'
  AND table_type = 'BASE TABLE';
```

En esta implementación el resultado esperado y reportado es 14. Las secuencias de IDENTITY no se cuentan como tablas de usuario.

7. Diagrama del modelo físico



Para mejor visualización por favor haga clic en el siguiente enlace:

https://drive.google.com/file/d/1xkG-8b0UmnUgl00jiP3Mzr-JZ5srlpy7/view?usp=sharing

8. Verificaciones realizadas

Se comprobaron las tablas y se hicieron las siguientes pruebas básicas en pgAdmin.

Comprobación	Resultado reportado / evidencia
Creación desde cero	El script se ejecutó y la consulta de inventario devolvió 14 tablas.
Inserción de estudiantes	Se aceptaron dos estudiantes de prueba con los datos obligatorios.
IDENTITY en deporte	INSERT sin id_deporte generó un identificador numérico.
UUID y fecha en publicacion	INSERT sin id_publicacion ni fecha_publicacion generó ambos valores.
Relación de seguimiento	Se aceptó una conexión entre dos estudiantes existentes.
Consulta con JOIN	Se recuperó una publicación junto con el nombre de su autor.
CHECK del semestre	La inserción con semestre 30 fue rechazada.
ROLLBACK	Las inserciones de la transacción de prueba se revirtieron sin eliminar las tablas.

.

9. Conclusiones

Esta etapa me permitió llevar a una base de datos real el modelo que venía trabajando desde el análisis de necesidades. No empecé nuevamente: tomé las cuatro tablas del foro y fui ampliando el código hasta completar las catorce. Eso me ayudó a entender que el modelo físico no es solo copiar el diagrama, sino decidir cómo se van a almacenar y proteger los datos.

Una de mis principales dificultades fue adaptar mi forma de trabajar en C++ a PostgreSQL. Me costó acostumbrarme a las comas dentro de CREATE TABLE y a entender que borrar el texto del editor no borra una tabla que ya fue creada. Al trabajar con Test, ALTER TABLE y el archivo SQL pude distinguir mejor entre las instrucciones guardadas y los cambios aplicados a la base de datos.

También comprendí la diferencia entre identificar un registro y generar su identificador. La PK garantiza la unicidad, mientras que IDENTITY o gen_random_uuid() ayudan a producir el valor. Cuando cambié esos tipos tuve que revisar las FK relacionadas, lo que hizo más clara la importancia de conservar la coherencia entre tablas.

Otro aprendizaje importante fue que CASCADE no debe utilizarse sin pensar en sus consecuencias. No es lo mismo eliminar una asociación personal que borrar un grupo donde participan otros estudiantes. Finalmente, las pruebas me mostraron que recibir un error ante un dato inválido puede ser una evidencia de que la restricción funciona. Como siguiente mejora del proyecto, ampliaría esas pruebas antes de conectar una aplicación y definiría con más detalle las reglas de formato y conservación del historial.

10. RECURSOS

Link al video de youtube: https://www.youtube.com/watch?v=h5runGPQh1g

Link al mismo video pero en google drive:https://docs.google.com/videos/d/1BxAao16m5_zYExClKuObPRvESCwt6terFtfcGFmlSqo/play?usp=sharing

Link GitHub: https://github.com/DanSPZ/bd1-redsocial-grupo11/tree/main/tarea3

Link de descarga del Script .sql:https://drive.google.com/file/d/1MX3Xhp2XEehM89__hWJm364UrfFcQkH2/view?usp=sharing

Link de la foto del diagrama: https://drive.google.com/file/d/1xkG-8b0UmnUgl00jiP3Mzr-JZ5srlpy7/view?usp=sharing